

KERNEL PROGRAMMING & PERFORMANCE

Implementação Getturnaround()

Análise do Escalonador e Modificações de Quantum no Kernel XV6

Motivação do Projeto

Entender as métricas de performance e o impacto das políticas de escalonamento no ciclo de vida dos processos.

| Principais Objetivos



Cálculo de TT

Implementar mecanismos para medir o **Turnaround Time (TT)** exato de cada processo no sistema.



Ajuste do Quantum

Modificar as configurações de interrupção do timer para alterar a fatia de tempo do Round Robin.



Estudo de Impacto

Analisar como diferentes valores de Quantum afetam o tempo médio de resposta e o overhead do sistema.

| Algoritmos e Mecanismos Base

Round Robin (RR)

Algoritmo preemptivo que utiliza alternância circular. Cada processo recebe uma fatia de tempo idêntica para uso da CPU.

O Quantum

Intervalo fixo que o processo possui antes de sofrer preempção. No XV6, este intervalo é ditado pelos ticks do timer.

Definição de Turnaround Time

Tempo total desde a criação até a morte do processo.

$$TT = T_{\text{fim}} - T_{\text{início}}$$

Inclui: Tempo em fila, execução e espera por recursos de E/S.

Implementação Técnica

Anotando o Ciclo de Vida do Processo

| Tracking do Ciclo de Vida

Anotação Temporal

Para o cálculo preciso, o Kernel deve registrar dois momentos críticos:

🍼 **Nascimento:** Captura de ticks no `allocproc()`.

💀 **Encerramento:** Captura de ticks no final da execução.

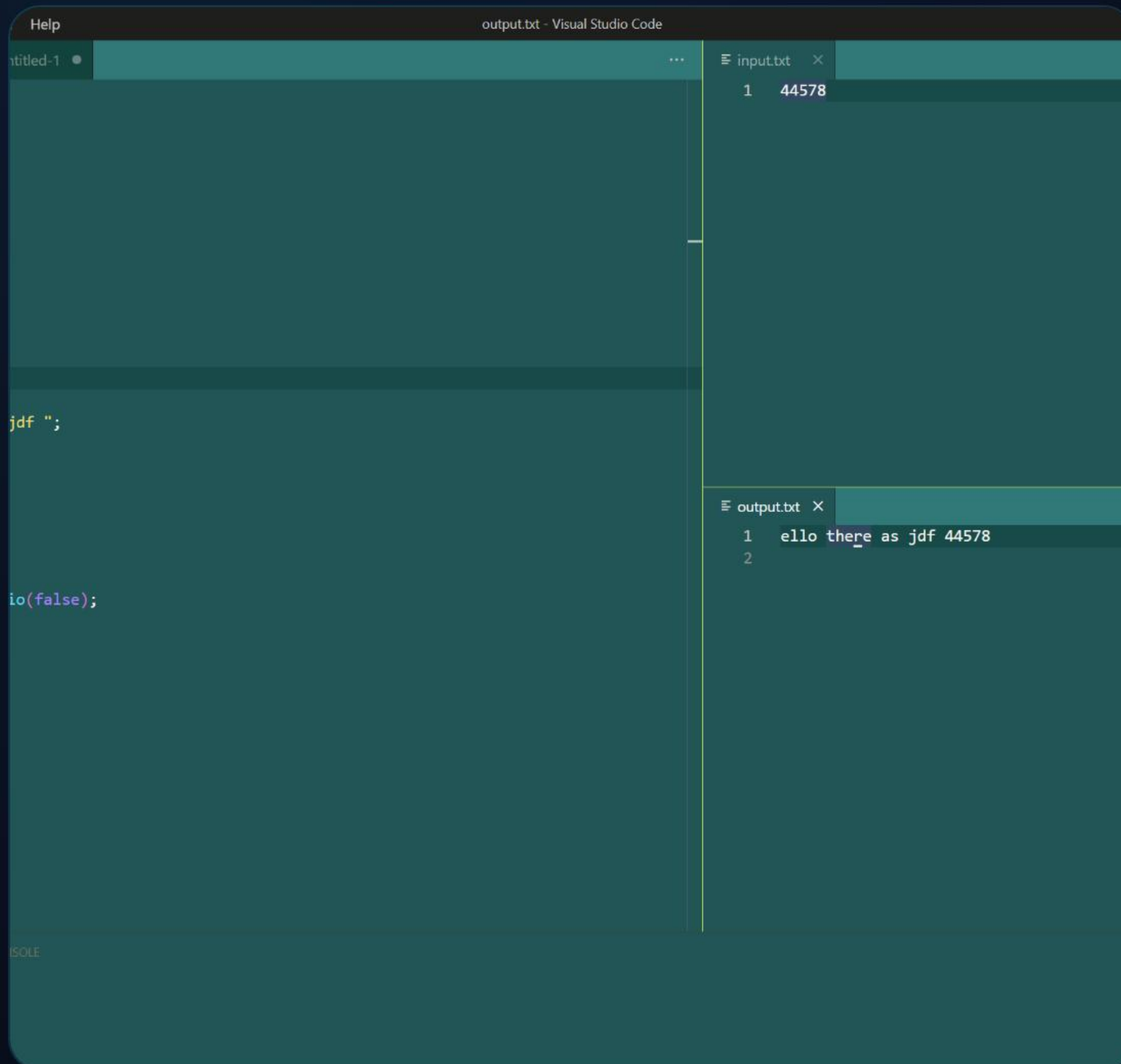
A função `getturnaround()` apenas realiza a aritmética final no retorno ao usuário.

OC
tial of Technology

Complete Guide to Trading Software Development in 2026



Modificação do Quantum



Alterações no start.c

O arquivo `start.c` é responsável por configurar o timer de hardware e as interrupções periódicas.

```
// Modificação no intervalo\nw_mstatus(r_mstatus() | MSTATUS_MIE);\ntimerinit(); // Configuração do Quantum
```

Reduzir este intervalo aumenta a frequência das interrupções, encurtando o Quantum efetivo do Round Robin.

| Registro da Chamada de Sistema

ARQUIVO	MUDANÇA NECESSÁRIA	RESPONSABILIDADE
<code>syscall.h</code>	<code>#define SYS_getturnaround 22</code>	Definição do número identificador.
<code>syscall.c</code>	<code>extern int sys_getturnaround(void);</code>	Mapeamento do ponteiro da função.
<code>sysproc.c</code>	<code>int sys_getturnaround(void) { ... }</code>	Lógica de negócio e acesso à struct <code>proc</code> .
<code>user.h</code>	<code>int getturnaround(void);</code>	Protótipo para chamadas em User Space.

| Impacto: Quantum Muito Grande

Turnaround Médio

Alto (Impacto FCFS)

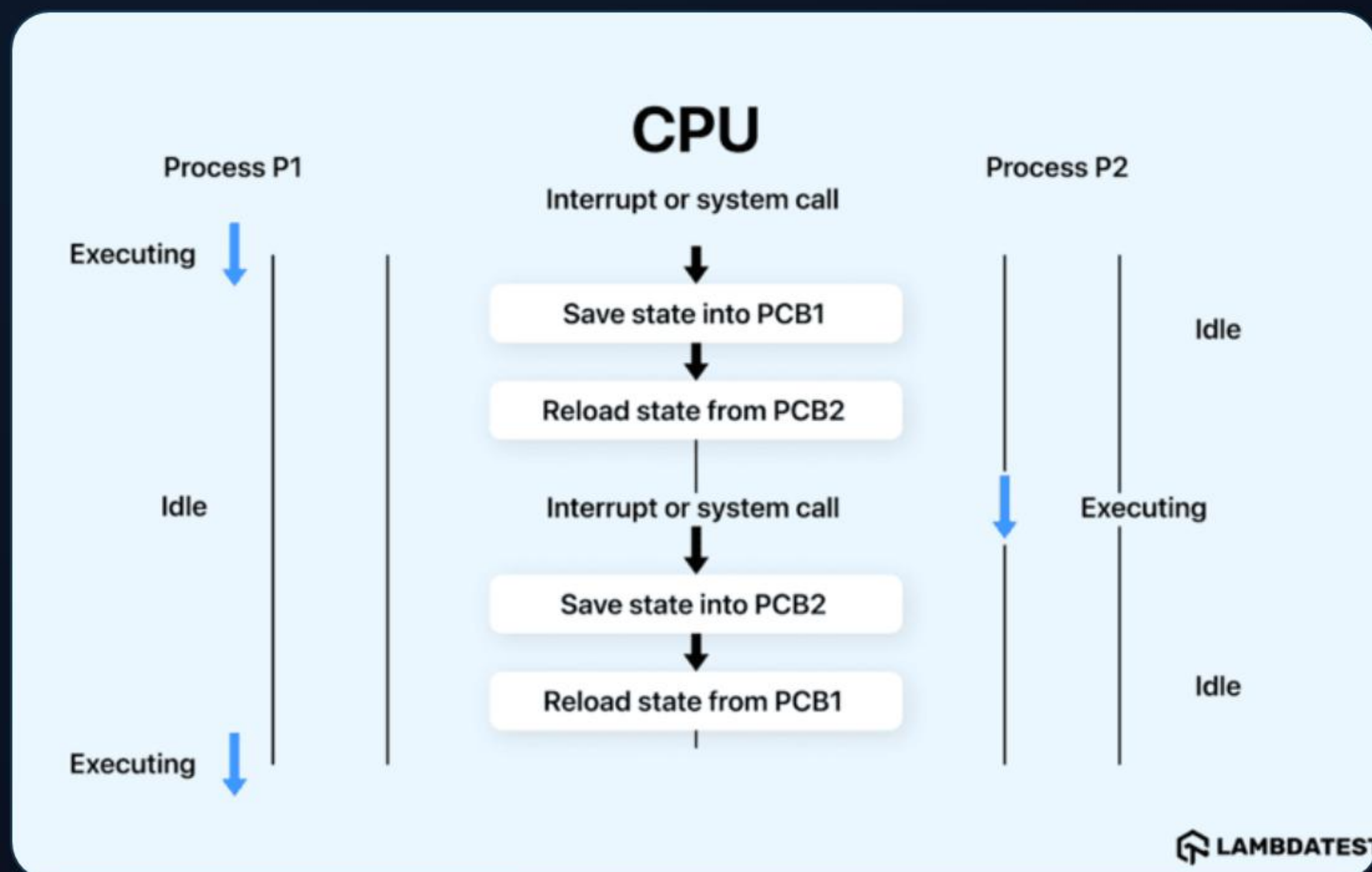
Justiça (Fairness)

Baixa

Se o Quantum exceder o tempo do processo mais longo, o RR se comporta como **FCFS (First-Come First-Served)**.

Processos curtos ficam presos atrás de longos, inflando o tempo total de permanência no sistema de forma drástica.

| Impacto: Quantum Muito Pequeno



O Problema do Overhead

Quantums pequenos dão a ilusão de execução simultânea, mas geram excesso de **Trocas de Contexto (Context Switches)**.

- Consumo de ciclos de CPU para salvar/carregar estados.
- O tempo útil de execução diminui proporcionalmente.
- Aumento do Turnaround médio devido ao peso administrativo do SO.

| ○ Equilíbrio Ideal

80%

A "Regra de Ouro" do Quantum

Para um escalonamento eficiente, cerca de **80% das rajadas de CPU** devem ser mais curtas que o Quantum.

Este equilíbrio garante que a maioria dos processos finalize ou bloqueie por E/S naturalmente, mantendo a responsividade sem sacrificar a CPU com trocas de contexto desnecessárias.

Dúvidas?

Implementação Getturnaround() e Tuning de Quantum no XV6

www.kernel-dev.edu | contact@sysadmin.io

| Image Sources



<https://www.tuvoc.com/wp-content/uploads/2025/12/Complete-guide-to-trading-software-development.png>

Source: www.tuvoc.com



<https://i.sstatic.net/mHIVc.png>

Source: stackoverflow.com



<https://assets.testmuai.com/resources/uploads/2025/02/State-Diagram-and-Steps-of-Context-Switching-Process-2.png>

Source: www.testmuai.com